ДОКУМЕНТАЦИЯ

Техническая документация проекта

Прогнозирование энергопотребления и стоимости электричества

---

1. Назначение и описание проекта

1.1 Целевая функция

Проект реализует систему прогнозирования двух ключевых показателей:

1. **Энергопотребление (кВт·ч)** — объём электроэнергии, потребляемой за день
2. **Стоимость электроэнергии (руб/кВт·ч)** — цена за единицу электроэнергии
3. **Итоговые расходы (рубли)** — произведение энергопотребления и цены

Система использует **линейную регрессию** (LinearRegression из scikit-learn) для поиска зависимостей между метеорологическими, календарными факторами и целевыми показателями.

1.2 Входные данные

Проект работает с синтетическими данными, включающими:

- **Временной период**: 100 дней (с 1 января 2021 года)
- **Температура воздуха**: от -10 до +10 °C (целые значения)
- **Энергопотребление**: от 200 до 500 кВт·ч (целые значения)
- **Цена электроэнергии**: от 4.5 до 6.5 руб/кВт·ч (с шумом ±0.1)

1.3 Целевая аудитория

- Студенты, изучающие основы машинного обучения
- Специалисты по анализу данных
- Разработчики систем прогнозирования
- Энергетические компании (для демонстрационных целей)

---

2. Архитектура решения

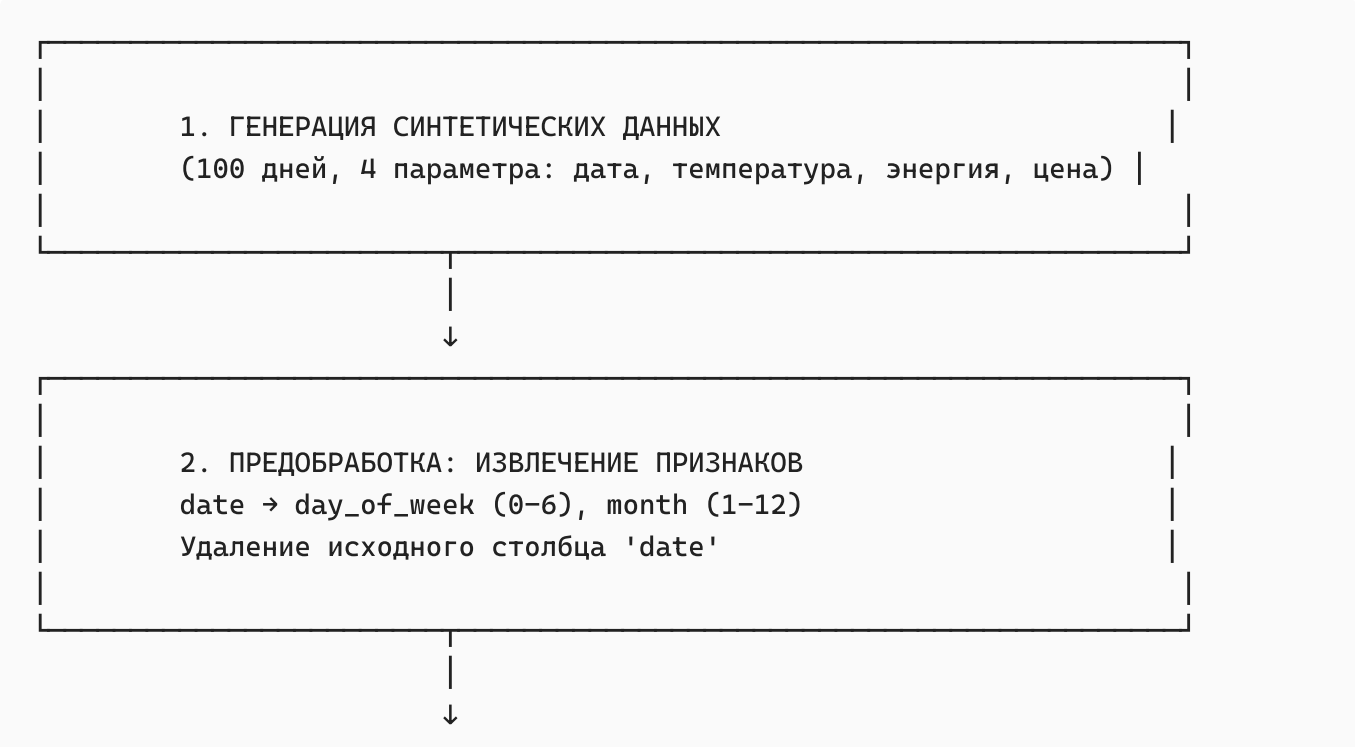
2.1 Стек технологий

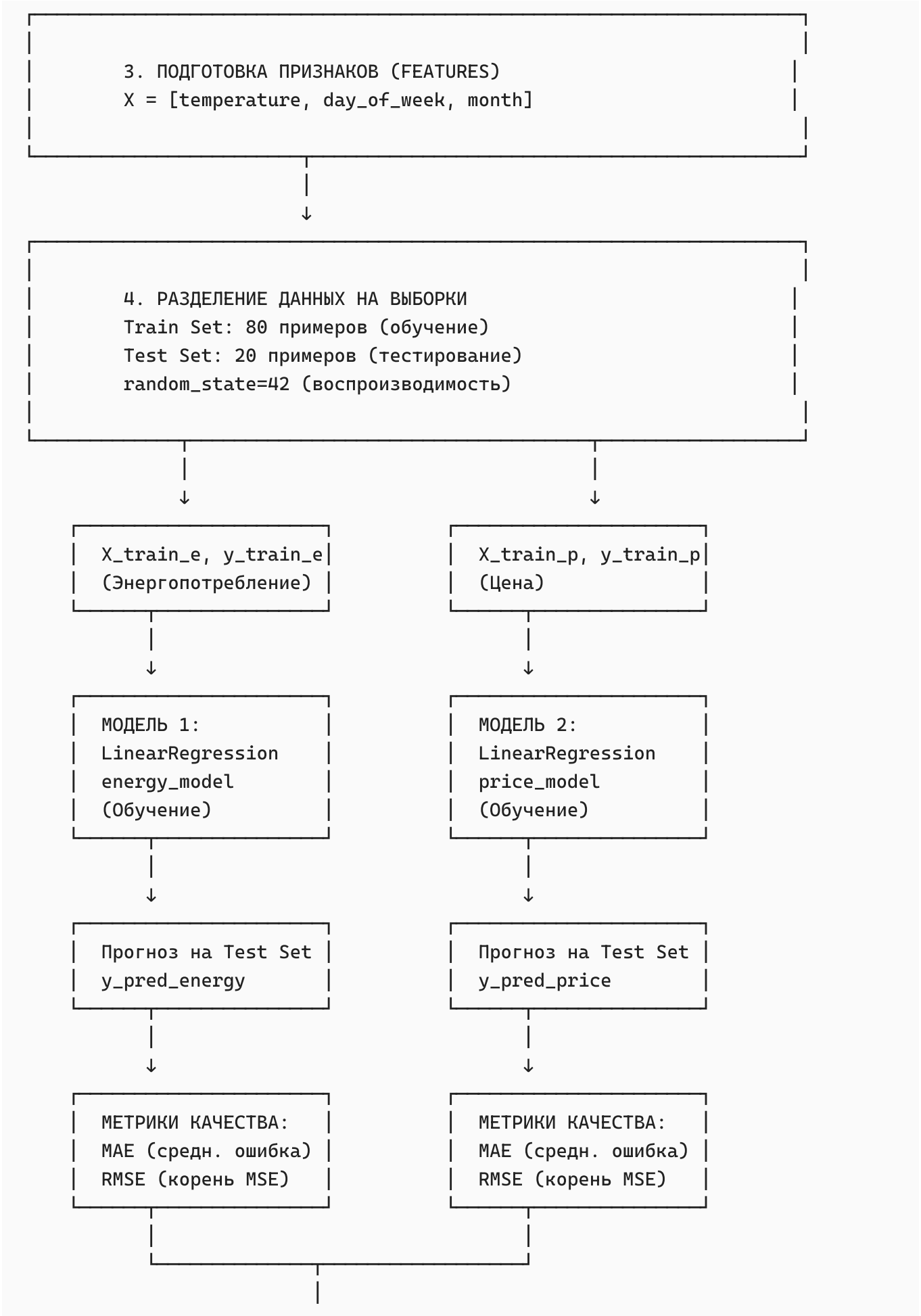
Компонент	Библиотека	Версия	Назначение
Обработка данных	pandas	1.3+	Работа с табличными данными (DataFrame)
Числовые вычисления	numpy	1.21+	Генерация массивов, математические операции
Машинное обучение	scikit-learn	0.24+	Линейная регрессия, разделение данных, метрики качества
Визуализация	matplotlib	3.4+	Построение графиков и диаграмм рассеяния
Статистическое отображение	seaborn	0.11+	Расширенные графики, гистограммы с KDE (сглаженное отображение распределения)

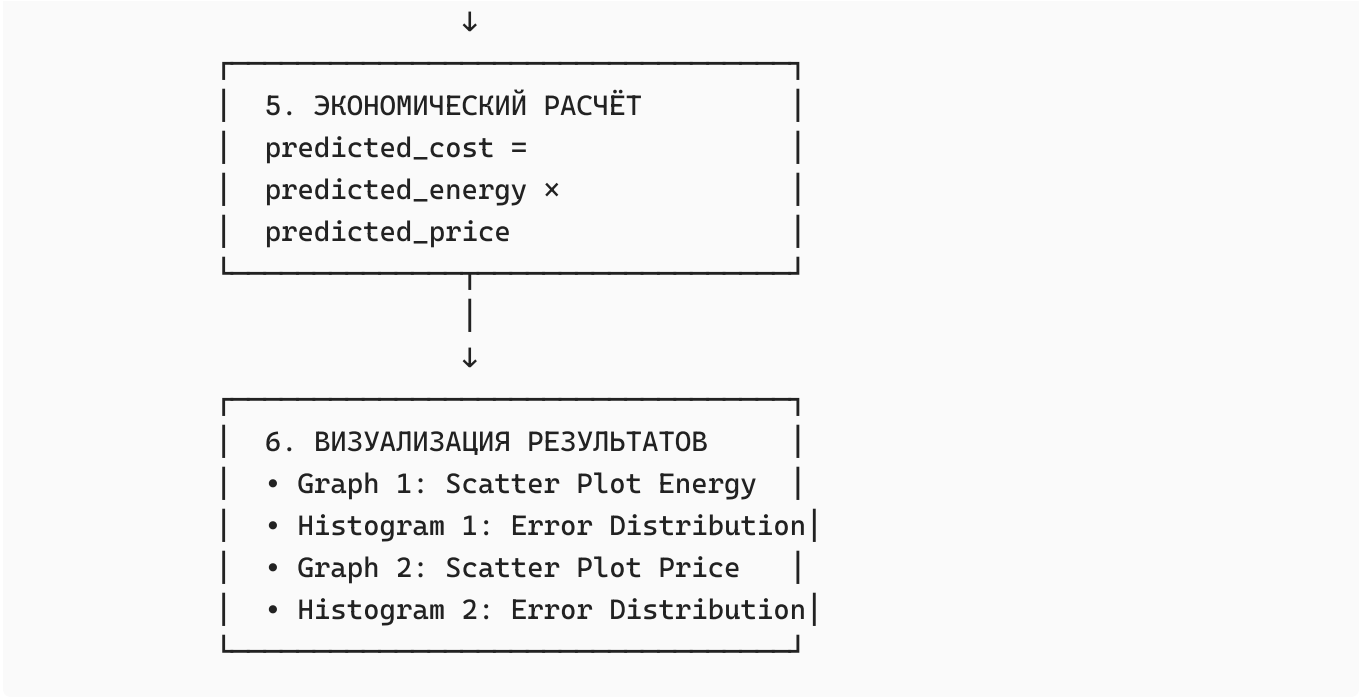
2.2 Архитектурная схема

Подробная архитектурная схема представлена на отдельной странице (см. раздел 2.3).

2.3 Полная архитектурная схема проекта




3. ПОДГОТОВКА ПРИЗНАКОВ (FEATURES)
X = [temperature, day_of_week, month]
4. РАЗДЕЛЕНИЕ ДАННЫХ НА ВЫБОРКИ
Train Set: 80 примеров (обучение)
Test Set: 20 примеров (тестирование)
random_state=42 (воспроизводимость)
X_train_e, y_train_e
(Энергопотребление)
X_train_p, y_train_p
(Цена)
МОДЕЛЬ 1:
LinearRegression
energy_model
(Обучение)
МОДЕЛЬ 2:
LinearRegression
price_model
(Обучение)
Прогноз на Test Set
y_pred_energy
Прогноз на Test Set
y_pred_price
МЕТРИКИ КАЧЕСТВА:
MAE (средн. ошибка)
RMSE (корень MSE)
МЕТРИКИ КАЧЕСТВА:
MAE (средн. ошибка)
RMSE (корень MSE)



---

3. Детальное описание кода

3.1 Блок 1: Импорт библиотек и генерация данных

```
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_absolute_error, mean_squared_error
import matplotlib.pyplot as plt
import seaborn as sns
```

Назначение: Подключение всех необходимых библиотек.

Генерация синтетического датасета:

```
data = {
    'date': pd.date_range(start='2021-01-01', periods=100, freq='D'),
    'temperature': np.random.randint(-10, 10, size=100),
    'energy_consumption': np.random.randint(200, 500, size=100),
}
df = pd.DataFrame(data)
df['price_per_kwh'] = np.linspace(4.5, 6.5, len(df)) + np.random.normal(0,
0.1, len(df))
```

Описание параметров:

- `pd.date_range()` — создаёт временной ряд из 100 последовательных дней
- `np.random.randint()` — генерирует целые числа в заданном диапазоне
- `np.linspace()` — равномерное распределение чисел от 4.5 до 6.5
- `np.random.normal()` — добавляет гауссовский шум для реалистичности

3.2 Блок 2: Предобработка данных (Feature Engineering)

```
df['day_of_week'] = df['date'].dt.dayofweek    # 0=понедельник, 6=воскресенье
df['month'] = df['date'].dt.month               # 1-12
df = df.drop('date', axis=1)                    # Удаляем исходную дату
```

Признаки для моделей:

- **day_of_week** — день недели (числовой признак 0-6)
- **month** — номер месяца (числовой признак 1-12)
- **temperature** — температура в градусах Цельсия

Почему удаляется дата: Дата — не числовой признак. Её разложили на компоненты (день недели, месяц), которые модель может использовать.

3.3 Блок 3: Модель 1 — Прогнозирование энергопотребления

```
X_energy = df[['temperature', 'day_of_week', 'month']]
y_energy = df['energy_consumption']

X_train_e, X_test_e, y_train_e, y_test_e = train_test_split(
    X_energy, y_energy, test_size=0.2, random_state=42
)

energy_model = LinearRegression()
energy_model.fit(X_train_e, y_train_e)
y_pred_energy = energy_model.predict(X_test_e)

mae_energy = mean_absolute_error(y_test_e, y_pred_energy)
rmse_energy = np.sqrt(mean_squared_error(y_test_e, y_pred_energy))
```

Параметры разделения:

- `test_size=0.2` — 20 примеров на тест, 80 на обучение
- `random_state=42` — фиксированное значение для воспроизводимости

Метрики качества:

- **MAE** (Mean Absolute Error) — средняя абсолютная ошибка в кВт·ч
- **RMSE** (Root Mean Squared Error) — корень из среднеквадратичной ошибки (штрафует большие ошибки)

3.4 Блок 4: Визуализация Модели 1

График 1: Реальные vs Прогнозируемые значения

```
plt.figure(figsize=(10, 6))
plt.scatter(y_test_e, y_pred_energy, label='Предсказанные значения')
plt.plot([y_test_e.min(), y_test_e.max()],
         [y_test_e.min(), y_test_e.max()],
         lw=2, label='Линия идеального предсказания')
plt.title('Реальные vs Предсказанные значения энергопотребления')
plt.xlabel('Реальное энергопотребление (кВт·ч)')
plt.ylabel('Предсказанное энергопотребление (кВт·ч)')
plt.legend()
plt.show()
```

Интерпретация:

- Если все точки лежат на диагональной линии → модель идеально предсказывает
- Если точки рассеяны вокруг линии → модель хорошая, но с небольшими ошибками
- Если облако точек далеко от линии → модель слабая

Гистограмма 1: Распределение ошибок

```
errors_energy = y_test_e - y_pred_energy
sns.histplot(errors_energy, kde=True)
plt.title('Распределение ошибок предсказания энергопотребления')
```

Назначение параметра `kde=True`: Добавляет плавную сглаживающую линию (KDE — Kernel Density Estimation) поверх столбиков гистограммы. KDE показывает общее распределение данных, сглаживая скачки столбцов.

Интерпретация:

- Если гистограмма сосредоточена около нуля → ошибки маленькие
- Если широко разбросана → ошибки большие
- Если смещена вправо → модель недооценивает; влево → переоценивает

- Если линия KDE имеет один чёткий пик → ошибки сосредоточены в одной области (хорошо)
- Если линия KDE имеет несколько пиков → есть разные типы ошибок (нужна оптимизация)

3.5 Блок 5: Модель 2 — Прогнозирование цены

Аналогично Модели 1, но для целевой переменной `price_per_kwh`:

```
X_price = df[['temperature', 'day_of_week', 'month']]
y_price = df['price_per_kwh']

X_train_p, X_test_p, y_train_p, y_test_p = train_test_split(
    X_price, y_price, test_size=0.2, random_state=42
)

price_model = LinearRegression()
price_model.fit(X_train_p, y_train_p)
y_pred_price = price_model.predict(X_test_p)

mae_price = mean_absolute_error(y_test_p, y_pred_price)
rmse_price = np.sqrt(mean_squared_error(y_test_p, y_pred_price))
```

Различие от Модели 1: Только целевая переменная (`y_price` вместо `y_energy`).

3.6 Блок 6: Экономический расчёт

```
df['predicted_energy'] = energy_model.predict(X_energy)
df['predicted_price'] = price_model.predict(X_price)
df['predicted_cost'] = df['predicted_energy'] * df['predicted_price']
```

Назначение: Вычисление прогнозируемых дневных расходов на электричество.

---

4. Математическая основа

4.1 Линейная регрессия

Модель ищет линейную зависимость:

$$\hat{y} = w_0 + w_1 \cdot \text{temperature} + w_2 \cdot \text{day_of_week} + w_3 \cdot \text{month}$$

где:

- $\hat{y}$ — предсказанное значение (энергия или цена)
- w_0, w_1, w_2, w_3 — коэффициенты, найденные при обучении
- temperature, day_of_week, month — входные признаки

4.2 Метрики качества

MAE (Mean Absolute Error) — Средняя абсолютная ошибка

Математическое определение:

$$\text{MAE} = \frac{1}{n}\sum_{i=1}^{n}|y_i - \hat{y}_i|$$

Простое объяснение:

- Берутся все ошибки: реальное значение минус предсказанное (`y_i - ŷ_i`)
- Берется абсолютное значение (модуль) каждой ошибки (чтобы -10 и +10 были одинаково «плохи»)
- Считается среднее значение всех ошибок

Пример:
Если MAE энергопотребления = 45 кВт·ч, значит модель ошибается в среднем на 45 кВт·ч. Это может быть недооценка на 45 или переоценка на 45.

Преимущества:

- Интуитивно понятна: результат в тех же единицах, что и целевая переменная
- Прямолинейная интерпретация: среднее отклонение прогноза

Недостатки:

- Не штрафует за большие ошибки. Одна ошибка на 1000 и сто ошибок на 10 дают одинаковый MAE

RMSE (Root Mean Squared Error) — Среднеквадратичная ошибка

Математическое определение:

$$\text{RMSE} = \sqrt{\frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2}$$

Простое объяснение:

- Берутся все ошибки: (`y_i - ŷ_i`)
- Каждая ошибка возводится в квадрат (большие ошибки становятся ещё больше)

- Считается среднее значение квадратов ошибок
- Берется квадратный корень (чтобы вернуться к исходным единицам)

Пример:
Если RMSE энергопотребления = 65 кВт·ч, это означает, что в среднем модель ошибается примерно на 65 кВт·ч. Но RMSE выше, чем MAE, потому что есть большие ошибки.

Преимущества:

- Сильно штрафует за большие ошибки (возведение в квадрат)
- Полезна, когда большие ошибки недопустимы (например, в критичных системах)

Недостатки:

- Результат зависит от наличия выбросов (одна огромная ошибка сильно влияет на RMSE)
- Менее интуитивна для интерпретации

Сравнение MAE и RMSE

Сценарий	MAE	RMSE	Интерпретация
Ошибки: 1, 2, 3	2.0	2.16	Примерно одинаковые, нет выбросов
Ошибки: 1, 2, 100	34.3	57.9	RMSE намного выше из-за большой ошибки
Ошибки: 5, 5, 5	5.0	5.0	Идентичны, когда все ошибки одинаковые

Когда использовать:

- **MAE**: Когда нужна простая интерпретация, ошибки примерно одинакового размера
- **RMSE**: Когда большие ошибки недопустимы и нужно их минимизировать

Практическое применение в проекте

```
# Расчёт MAE и RMSE для энергопотребления
mae_energy = mean_absolute_error(y_test_e, y_pred_energy)
rmse_energy = np.sqrt(mean_squared_error(y_test_e, y_pred_energy))

print(f"MAE энергопотребления: {mae_energy:.2f} кВт·ч")
print(f"RMSE энергопотребления: {rmse_energy:.2f} кВт·ч")

# Интерпретация:
# Если MAE = 42 кВт·ч, RMSE = 58 кВт·ч
```

```
# → Есть несколько больших ошибок (RMSE > MAE)
# → В среднем ошибка около 42 кВт·ч
```

---

5. Использование проекта

5.1 Запуск программы

```
python 11.py
```

Вывод в консоль:

```
Прогноз энергопотребления:
MAE: XX.XX
RMSE: XX.XX

Прогноз цены электроэнергии:
MAE: XX.XX
RMSE: XX.XX

Экономический прогноз (первые 5 строк):
   predicted_energy  predicted_price  predicted_cost
0            XXX.XX            X.XX           XXXX.XX
...
```

5.2 Интерпретация результатов

Метрика	Хорошее значение	Плохое значение
MAE энергии	< 50 кВт·ч	> 150 кВт·ч
RMSE энергии	< 60 кВт·ч	> 200 кВт·ч
MAE цены	< 0.3 руб	> 1.0 руб
RMSE цены	< 0.4 руб	> 1.5 руб

Пороги условные — зависят от масштаба данных.

---

6. Возможные улучшения

6.1 Краткосрочные улучшения

1. **Добавить больше признаков**:
   - Праздничные дни (boolean flag)
   - Влажность воздуха
   - Скорость ветра
   - Солнечная активность
2. **Использовать нелинейные модели**:
   - RandomForestRegressor (ансамбль деревьев)
   - GradientBoostingRegressor (более точный, медленнее)
   - SVR (метод опорных векторов)
3. **Кросс-валидация**:

```
from sklearn.model_selection import cross_val_score
scores = cross_val_score(energy_model, X_energy, y_energy, cv=5)
```

6.2 Среднесрочные улучшения

- Работа с реальными данными (Open Data платформы)
- Обработка пропущенных значений (NaN)
- Нормализация признаков (StandardScaler, MinMaxScaler)
- Отбор наиболее значимых признаков (feature selection)

6.3 Долгосрочные улучшения

- Развёртывание в production (Flask/FastAPI API)
- Использование временных рядов (ARIMA, Prophet)
- Глубокое обучение (LSTM, Transformer)
- Интеграция с системой мониторинга и алертов

---

7. Возможные проблемы и решения

Проблема	Причина	Решение
MAE/RMSE слишком большие	Недостаточно признаков	Добавить больше входных данных
Модель переобучена	Малый датасет	Увеличить объём данных, использовать регуляризацию

Проблема	Причина	Решение
Модель недообучена	Сложность задачи	Попробовать нелинейные модели
Значительная корреляция между признаками	Мультиколлинеарность	Удалить коррелирующие признаки, использовать PCA

---

8. Зависимости и установка

8.1 Требования

```
pandas>=1.3.0
numpy>=1.21.0
scikit-learn>=0.24.0
matplotlib>=3.4.0
seaborn>=0.11.0
```

8.2 Установка

```
pip install pandas numpy scikit-learn matplotlib seaborn
```

---

9. Практические примеры использования

9.1 Пример 1: Прогноз для конкретного дня

```
# Предположим, день недели=2 (среда), месяц=6 (июнь), температура=15°C
new_data = np.array([[15, 2, 6]])
predicted_energy = energy_model.predict(new_data)[0]
predicted_price = price_model.predict(new_data)[0]
predicted_cost = predicted_energy * predicted_price

print(f"Прогноз на среду, июнь, 15°C:")
print(f"Энергопотребление: {predicted_energy:.2f} кВт·ч")
print(f"Цена: {predicted_price:.2f} руб/кВт·ч")
print(f"Итоговые расходы: {predicted_cost:.2f} руб")
```

9.2 Пример 2: Анализ влияния температуры

```
temperatures = np.arange(-10, 15, 1)
predictions = []

for temp in temperatures:
    X_sample = np.array([[temp, 3, 6]])  # Фиксируем день и месяц
    pred = energy_model.predict(X_sample)[0]
    predictions.append(pred)

plt.plot(temperatures, predictions)
plt.xlabel('Температура (°C)')
plt.ylabel('Энергопотребление (кВт·ч)')
plt.title('Влияние температуры на энергопотребление')
plt.show()
```

---

10. Заключение

Данный проект демонстрирует основные этапы работы с машинным обучением:

✓ Подготовка и обработка данных
✓ Разделение на обучающую и тестовую выборки
✓ Обучение моделей
✓ Оценка качества моделей
✓ Визуализация результатов
✓ Практическое применение для прогнозирования

Проект может служить основой для более сложных приложений в энергетике и управлении ресурсами.